LangGraph Documentation

Knowledge Base Reference for Retrieval-Augmented Generation Systems

Introduction

LangGraph is a framework for building stateful AI workflows using graph-based execution. It enables routing, memory, tools, and agentic behavior.

Core Concepts

The primary concepts are State, Nodes, Edges, Conditional Edges, Entry Points, Finish Points, and Graph Compilation.

State Management

State is the shared object passed between nodes. Common fields include query, route, model, context, answer, sources, and session_id.

Nodes

Nodes are Python functions that process and update state. Examples include router nodes, retrieval nodes, and tool nodes.

Conditional Routing

Conditional edges allow workflows to dynamically choose execution paths based on state values.

Multi-LLM Orchestration

A model-router can dynamically choose Mistral, Qwen, Gemma, or Phi based on task requirements.

RAG Workflows

RAG combines retrieval from a vector database with LLM generation to produce grounded responses.

ChromaDB

ChromaDB stores embeddings and supports semantic search over documents.

Memory

Persistent memory enables conversation continuity and session-aware interactions.

Tool Calling

LangGraph nodes can invoke calculators, APIs, search systems, databases, and custom tools.

Production Systems

Production-ready systems should include logging, monitoring, evaluation, and robust error handling.

Estudio PolyMind Example

LangGraph orchestrates routing, model selection, RAG retrieval, tool execution, and memory updates in Estudio PolyMind.

Example Workflow

User Query → Router → Model Router → Direct LLM / RAG / Tool → Memory Update → Response